

HERALD COLLEGE KATHMANDU

Week 4

CI/CD with GitHub Actions

Automating build, push, and deploy for your MERN app · Tutorial 2 hrs · Workshop 2.5 hrs · Phase 2

Last week you wrote scripts and ran them by hand. This week a machine runs them for you — automatically, every time you push code. You will learn what CI/CD is, write your first GitHub Actions workflow, then build two real pipelines: one for the backend and one for the frontend, each of which builds an image, pushes it to Docker Hub, and deploys it to your EC2 server — with your SSH key kept safe, never in the repository.

By the end of this week you can

- explain what CI/CD is and why it matters;
- read and write a GitHub Actions workflow file;
- store credentials as GitHub Secrets and use them in a workflow, never hardcoding them;
- build a pipeline that builds an image and pushes it to Docker Hub on every push;
- extend that pipeline to deploy the image onto EC2 over SSH, using a securely-stored key;
- build separate pipelines for the frontend and backend, and diagnose the common failures.

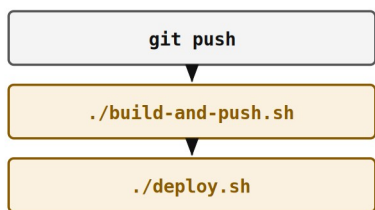
1 What Is CI/CD

CI/CD stands for **Continuous Integration** and **Continuous Deployment**. Stripped of jargon, it means one thing: the steps between “I changed some code” and “it is live on the server” happen automatically, triggered by your `git push`, with no manual steps.

Continuous Integration is the build-and-test half: every time code is pushed, it is automatically built and checked, so problems are caught immediately rather than days later. **Continuous Deployment** is the delivery half: once the build passes, it is automatically shipped to where it runs.

You already have all the pieces from last week — the build, the push, the SSH deploy. The only thing missing is something to run them for you when you push. That something is **GitHub Actions**.

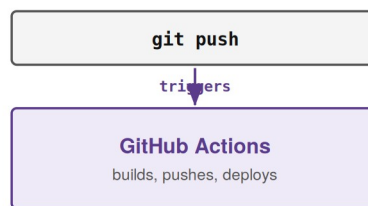
LAST WEEK — YOU RUN THE SCRIPTS



You remember to run each script, in order, by hand. Every release.

Forget one → stale or broken deploy.

THIS WEEK — GITHUB DOES IT



You push code. That is the whole job.

The pipeline runs itself, the same way, every single time.

Figure 1. Last week you ran the scripts by hand. This week a push to GitHub runs the whole pipeline for you.

2 GitHub Actions Basics

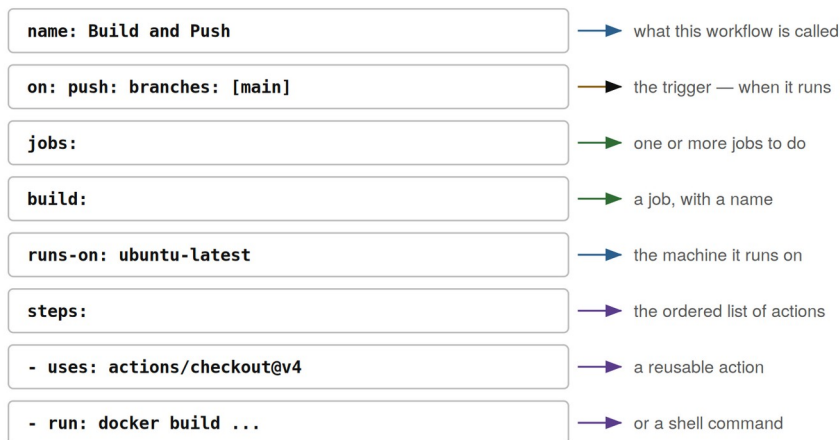
GitHub Actions is CI/CD built into GitHub. You describe what should happen in a **workflow** file, commit it to your repository, and GitHub runs it automatically on the trigger you choose — typically a push.

2.1 Where workflows live

A workflow is a YAML file inside a special folder in your repository: `.github/workflows/`. Any `.yml` file there is picked up automatically. You can have several — this week you will have two, one per part of the app.

2.2 The anatomy of a workflow

Every workflow has the same shape: a name, a trigger (on), and one or more jobs, each made of steps. A step either **uses** a ready-made action or **runs** a shell command — the same commands you already know.



A workflow is a YAML file in `.github/workflows/`. It says: on this trigger, run these jobs, each a list of steps.

Figure 2. The parts of a workflow file. Read top to bottom: name, trigger, jobs, steps.

2.3 Your first workflow

Create `.github/workflows/hello.yml` in your repository:

```
.github/workflows/hello.yml
name: Hello

on: push          # run on every push

jobs:
  greet:
    runs-on: ubuntu-latest    # a fresh Linux machine
    steps:
      - name: Say hello
        run: echo "Hello from GitHub Actions"
```

Commit and push it. Open the **Actions** tab of your repository on GitHub and you will see the workflow run, with the green output of your echo. Every push from now on triggers it.

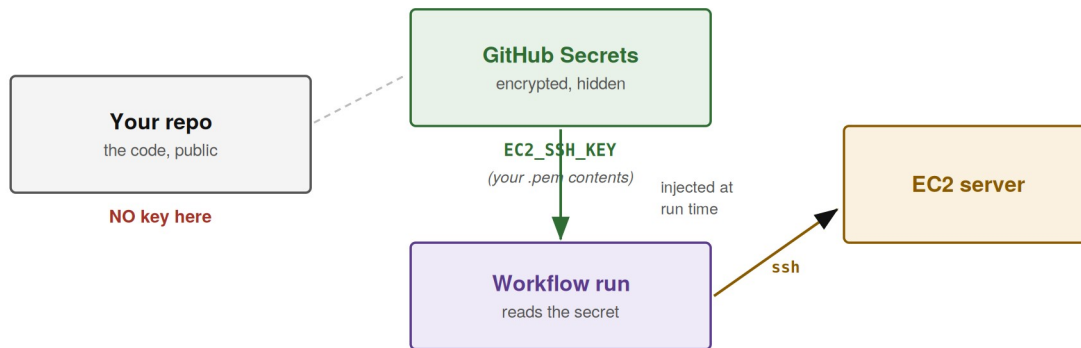
KEY IDEA

A workflow runs on a **fresh, empty machine** that GitHub provides (`runs-on: ubuntu-latest`). It has none of your files until you check them out, and it disappears when the job ends. Anything the job needs — your code, your credentials — must be brought in explicitly. That is what the next sections are about.

3 Secrets: Keeping Credentials Safe

A pipeline needs your Docker Hub password and your EC2 SSH key. These must never be written in a workflow file — the file lives in your repository, where anyone with access (and everyone, if it is public) could read them. This is the same rule from last week: credentials never go in scripts.

GitHub solves this with **Secrets**: encrypted values stored in your repository settings, hidden from everyone, and injected into a workflow only while it runs. This is GitHub's equivalent of a secure credential store — if you have used GitLab, it is the same idea as its secure files and CI/CD variables.



The .pem key lives in GitHub Secrets, never in the repository. The workflow reads it at run time to reach EC2.

Figure 3. Secrets live in repository settings, never in the code. The workflow reads them at run time.

3.1 Adding your secrets

In your repository, go to Settings → Secrets and variables → Actions, and add these. You will use all of them in the pipelines that follow.

Secret name	What to put in it
DOCKERHUB_USERNAME	Your Docker Hub username
DOCKERHUB_TOKEN	A Docker Hub access token (create at hub.docker.com → Account → Security)
EC2_HOST	Your EC2 instance's public IP address
EC2_USER	The server login name, ec2-user
EC2_SSH_KEY	The entire contents of your .pem key file

For EC2_SSH_KEY, open your .pem file in a text editor and copy everything — from -----BEGIN to -----END, inclusive — and paste it as the secret's value. The file itself never goes near your repository.

WHY A TOKEN, NOT YOUR PASSWORD?

For Docker Hub we store an **access token**, not your account password. A token can be limited in scope and revoked on its own if it leaks, without you having to change your password everywhere. Using tokens for automation instead of real passwords is standard practice.

3.2 Reading a secret in a workflow

Inside a workflow, a secret is read with `${{ secrets.NAME }}`. GitHub substitutes the value at run time and automatically masks it in the logs, so it never appears even if a step prints it.

with:

```
username: ${ secrets.DOCKERHUB_USERNAME }}
password: ${ secrets.DOCKERHUB_TOKEN }}
```

4 Pipeline 1: Build and Push the Backend

Now the first real pipeline. It does automatically what your `build-and-push.sh` did by hand: build the backend image and push it to Docker Hub — but triggered by a push, and only when the backend code changes. We build it in two stages: first writing the commands directly in the workflow so you see exactly what runs, then replacing them with the script you already wrote.

4.1 First, the commands written into the workflow

A step can **run** any shell command with `run:`. So the most direct way to build and push is to write the same commands you know straight into the workflow. Create `.github/workflows/backend.yml`:

[.github/workflows/backend.yml](#) — commands inline

```
name: Backend CI/CD

on:
  push:
    branches: [main]
    paths: ['server/**'] # only when backend code changes

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4 # get your code

      - name: Log in to Docker Hub
        run: echo "${ secrets.DOCKERHUB_TOKEN }}" |
            docker login -u "${ secrets.DOCKERHUB_USERNAME }}" --password-stdin

      - name: Build the image
        run: docker build -t ${ secrets.DOCKERHUB_USERNAME }}/mern-server:latest
            ./server

      - name: Push the image
        run: docker push ${ secrets.DOCKERHUB_USERNAME }}/mern-server:latest
```

Every `run:` line is a command you have typed before. The login uses `--password-stdin` so the token is piped in rather than typed — the non-interactive form of the `docker login` from Week 3. This works, and it is worth writing once to see the whole pipeline in plain commands.

WHAT IS PATHS FOR?

The `paths: [ 'server/**' ]` line means this workflow only runs when files under `server/` change. That way a frontend-only change does not needlessly rebuild the backend. The frontend workflow will do the same for `client/`.

4.2 Then, use the script you already wrote

Look at those three `run:` steps: they are exactly the contents of `build-and-push.sh` from Week 3. Repeating them in the workflow duplicates code you already have. The better approach — and the real lesson of this section — is to keep the logic in the script and have the workflow just **call it**.

First, move your script to a `scripts/` folder at the repo root so both pipelines can share it, and generalise it to read its settings from the environment instead of hardcoding them. The workflow will supply those values from your secrets.

scripts/build-and-push.sh

```
#!/bin/bash
set -euo pipefail

# settings arrive from the environment (the workflow provides them)
FULL_NAME="$DOCKERHUB_USERNAME/$IMAGE:latest"

# log in without a prompt, using the token piped in
echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USERNAME" --password-stdin

echo "Building $FULL_NAME from $CONTEXT ..."
docker build -t "$FULL_NAME" "$CONTEXT"

echo "Pushing $FULL_NAME ..."
docker push "$FULL_NAME"
```

Now the whole build-and-push job is a single step that runs the script, with the settings passed in through `env:`. Replace the three inline steps with this:

.github/workflows/backend.yml — calling the script

```
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Build and push
        run: bash scripts/build-and-push.sh
        env:
          DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
          DOCKERHUB_TOKEN: ${ secrets.DOCKERHUB_TOKEN }
          IMAGE: mern-server
          CONTEXT: ./server
```

The `env:` block is the bridge: it takes your secrets and settings and hands them to the script as environment variables, which is exactly what the script reads. We invoke it with `bash scripts/...` so it runs regardless of the file's executable bit, which can be lost on Windows.

WHY CALL THE SCRIPT INSTEAD OF INLINING?

The same script now works on your machine *and* in the pipeline — one place to fix a bug, one behaviour everywhere. And, as the next section shows, the frontend pipeline reuses the very same script with different values. Writing logic once and calling it from many places is a core habit; a workflow that just wires secrets to a script is easier to read and trust than one with the logic buried inside it.

5 Pipeline 1, Continued: Deploy to EC2

Building and pushing is the CI half. Now the CD half: after the image is on Docker Hub, connect to EC2 and deploy it. This is your `deploy.sh` from Week 3 — the one that SSHes from wherever it runs — now running on the GitHub runner instead of your laptop, with the key coming from a secret instead of a local `.pem` file.

5.1 The deploy script, reading the key from the environment

Your Week 3 `deploy.sh` read the key from a local file. The only change for CI is where the key comes from: the workflow puts the `EC2_SSH_KEY` secret into the environment, and the script writes it to a temporary file to use, then deletes it. Everything else — the `ssh` with `accept-new`, the `pull / stop / rm / run` — is unchanged.

scripts/deploy.sh

```
#!/bin/bash
set -euo pipefail

FULL_NAME="$DOCKERHUB_USERNAME/$IMAGE:latest"

# write the key from the environment to a temp file, lock it down
echo "$EC2_SSH_KEY" > key.pem
chmod 400 key.pem

# run the deploy commands on EC2 over SSH
ssh -o StrictHostKeyChecking=accept-new -i key.pem \
  "$EC2_USER@$EC2_HOST" "
    docker pull $FULL_NAME
    docker stop $CONTAINER 2>/dev/null || true
    docker rm $CONTAINER 2>/dev/null || true
    docker run -d --name $CONTAINER \
      --restart always -p $PORT:$PORT $FULL_NAME
  "

rm -f key.pem # never leave the key lying around
```

Add a second job to `backend.yml` that checks out the code and runs the deploy script, passing the connection details and settings from your secrets:

.github/workflows/backend.yml — deploy job (below the first)

```
deploy:
  needs: build-and-push # only after the image is pushed
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4

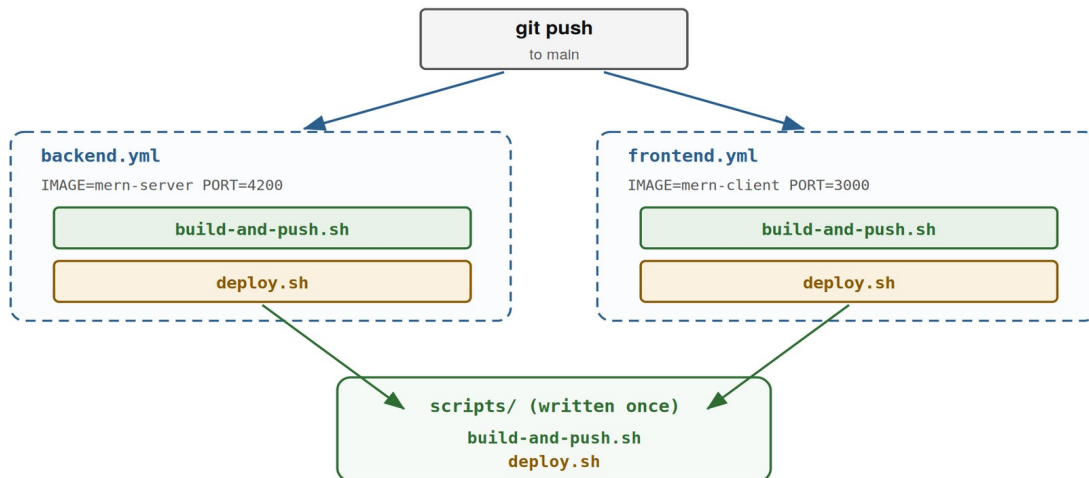
    - name: Deploy to EC2
      run: bash scripts/deploy.sh
      env:
        DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
        IMAGE: mern-server
        CONTAINER: mern-server
        PORT: 4200
        EC2_HOST: ${ secrets.EC2_HOST }
        EC2_USER: ${ secrets.EC2_USER }
        EC2_SSH_KEY: ${ secrets.EC2_SSH_KEY }
```

Two things worth noticing:

- `needs: build-and-push` — makes the deploy job wait until the build-and-push job has finished successfully. Without it the two jobs run at the same time, and you might deploy an image that has not been pushed yet.
- The key never touches your repository. It lives in the `EC2_SSH_KEY` secret, is written to a temporary file only for the moment the script runs on the runner, and is deleted immediately after. Anyone reading your code sees the script and the `env: wiring`, but none of the credentials.

6 Pipeline 2: The Frontend

Here the reuse pays off. The frontend needs the same build-push-deploy pipeline — and it can call the **exact same two scripts**, only with different values. You write no new script logic at all; you write a second workflow that points the shared scripts at the `client` folder and its port.



Both workflows call the same two scripts, each passing its own `IMAGE`, `CONTEXT`, `CONTAINER`, and `PORT`.

Figure 4. Two workflows share one pair of scripts, each passing its own values — build once, reuse everywhere.

Create `.github/workflows/frontend.yml`. Notice it is structurally identical to the backend — only the `env:` values change:

```

.github/workflows/frontend.yml
name: Frontend CI/CD

on:
  push:
    branches: [main]
    paths: ['client/**']      # only when frontend code changes

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build and push
        run: bash scripts/build-and-push.sh
    env:
      DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }
      DOCKERHUB_TOKEN: ${ secrets.DOCKERHUB_TOKEN }
      IMAGE: mern-client      # <- different image
      CONTEXT: ./client       # <- different folder

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to EC2
        run: bash scripts/deploy.sh
    env:

```

```
DOCKERHUB_USERNAME: ${ secrets.DOCKERHUB_USERNAME }}
IMAGE: mern-client      # <- different image
CONTAINER: mern-client
PORT: 3000              # <- different port
EC2_HOST: ${ secrets.EC2_HOST }}
EC2_USER: ${ secrets.EC2_USER }}
EC2_SSH_KEY: ${ secrets.EC2_SSH_KEY }}
```

Not a single new command — only four values differ (IMAGE, CONTEXT, CONTAINER, PORT). The two scripts you wrote for the backend drive the frontend too. That is the whole point of keeping the logic in scripts: the pipelines become thin wiring around code you already trust.

7 The Whole Thing, Working

You now have a complete CI/CD setup. To ship a change to the backend: edit a file under `server/`, commit, and `git push`. That is the entire deployment process. GitHub builds the image, pushes it to Docker Hub, connects to EC2, and runs the new version — while you watch the progress in the Actions tab.

A frontend change is the same, under `client/`. Each pipeline runs only when its own part changes, builds and publishes its own image, and deploys to the same server on its own port. Open `http://<EC2_PUBLIC_IP>:3000` and your app is live — shipped entirely by a `git push`.

HOW FAR YOU HAVE COME

Week 2 you built images by hand. Week 3 you scripted the build, push, and deploy. This week those scripts became a pipeline that runs itself. That progression — by hand, then scripted, then automated — is the core movement of DevOps, and you have now done it end to end on a real application.

8 Exercises

Each exercise gives you a problem to reproduce, then asks you to fix it. Use the material above and the GitHub Actions reference at the end.

A The workflow that never runs

PROBLEM

Put your workflow file in `workflows/backend.yml` (at the repo root), not `.github/workflows/backend.yml`. Push it. Nothing happens — the Actions tab shows no run at all.

YOUR TASK

Explain why GitHub ignored the file, and put it where it will run.

Hint. Section 2.1. GitHub only looks in one specific folder for workflows. Is your file in it?

B The build that cannot log in

PROBLEM

Write the backend workflow, but reference the secret as `${{ secrets.DOCKER_TOKEN }}` when the secret you created is named `DOCKERHUB_TOKEN`. Push.

The login step fails with an authentication error, even though your token is correct.

YOUR TASK

Find the mismatch and fix it.

Hint. A secret reference must match the secret's name exactly. A secret that does not exist expands to an empty value — so the login runs with a blank password.

C The pipeline that leaks the key

PROBLEM

A teammate, in a hurry, pastes the contents of the `.pem` file directly into `scripts/deploy.sh` instead of reading it from `EC2_SSH_KEY`, and commits it.

It works — the deploy succeeds. Explain why this is a serious problem anyway.

YOUR TASK

Describe what has gone wrong from a security standpoint, and correct it so the key is not exposed.

Hint. Section 3 and 5. The script is committed to the repository — who can read it? Once a private key is committed to git history, is deleting the line later enough?

D The deploy that runs too early

PROBLEM

Remove `needs: build-and-push` from the deploy job and push.

Sometimes the deploy works; sometimes it pulls an old image or fails to find the new one. It is inconsistent.

YOUR TASK

Explain why the result is unpredictable, and make the deploy reliably run after the push.

Hint. Section 5. Without `needs`, when do the two jobs run relative to each other? Can you deploy an image that has not finished pushing?

E The workflow that rebuilds everything

PROBLEM

Remove the `paths : filter` from both workflows. Make a one-line change to the frontend and push. Both pipelines run — the backend rebuilds and redeploys even though no backend code changed.

YOUR TASK

Explain why both ran, and make each pipeline run only when its own code changes.

Hint. Section 4. What does the `paths` filter do, and which folder should each workflow watch?

F The script that builds the wrong thing

PROBLEM

In the frontend workflow, call `build-and-push.sh` but forget the `env: block` — or leave `IMAGE` and `CONTEXT` set to the backend's values.

The job runs, but it builds the server image, or fails with an unbound-variable error from the script.

YOUR TASK

Explain how the script gets its settings, and fix the workflow so it builds the client.

Hint. Section 4.2. The script reads `IMAGE` and `CONTEXT` from the environment; `set -u` makes an unset variable an error. What must the `env: block` provide, and to which values?

9 When It Goes Wrong

Symptom	Cause and fix
Workflow does not run at all	The file is not in <code>.github/workflows/</code> , or the trigger does not match. Check the folder and the <code>on:</code> branch.
Login / auth error on Docker Hub	Secret name mismatch, or an expired token. Check <code>\${{ secrets.NAME }}</code> matches exactly, and that the token is valid.
ssh: handshake failed / permission denied	The <code>EC2_SSH_KEY</code> secret is incomplete or wrong. Re-copy the whole <code>.pem</code> including the <code>BEGIN</code> and <code>END</code> lines.
Script fails: unbound variable	A value the script expects was not passed. Check the step's <code>env:</code> block provides <code>IMAGE</code> , <code>CONTEXT</code> , <code>PORT</code> , etc. (Section 4.2).
Deploy job pulls an old image	The deploy ran before the push finished. Add <code>needs: build-and-push</code> to the deploy job.
Both pipelines run on any change	Missing paths filter. Add paths: <code>['server/**']</code> and <code>['client/**']</code> to the respective workflows.
App unreachable after a successful deploy	The EC2 security group does not allow the port. Add an inbound rule (Week 3, Section 4).

10 Before Next Week

Check you can do each of these without looking anything up.

1. Explain what CI and CD each mean, in one sentence.
2. Write a minimal workflow with a name, a push trigger, and one step, and see it run.
3. Store a value as a secret and read it in a workflow with `${{ secrets.NAME }}`.
4. Explain why the SSH key belongs in Secrets and not in the workflow file.
5. Push a code change and watch it build, push, and deploy with no manual steps.

Looking ahead: your pipeline deploys by replacing a single container. Real systems need to do this without dropping traffic, across many machines — which is where load balancing and orchestration come in, the direction the rest of the course takes.

Reference

- GitHub Actions docs — docs.github.com/actions
- Using secrets — docs.github.com/actions/security-guides/using-secrets-in-github-actions
- Passing values with `env` — docs.github.com/actions/learn-github-actions/variables
- Docker Hub access tokens — docs.docker.com/security/for-developers/access-tokens/